One frame, checked whole

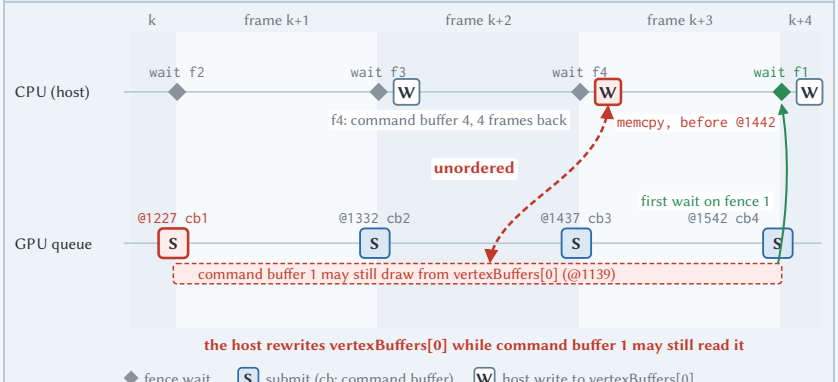
1 Host code, as shipped (SDL3)

```
// one set of vertex buffers for all frames
VULKAN_Buffer vertexBuffers[256];

// every frame (4 swapchain images,
// 4 command buffers, 4 fences)
cur = (cur + 1) % swapchainImageCount;
vkWaitForFences(..., &fences[cur], ...);
// waits only for the command buffer
// about to be reused, 4 frames back
vkAcquireNextImageKHR(...);
SDL_memcpy(vertexBuffers[0].mapped,
           vertices, size);   // BUG
// record draws reading vertexBuffers[0]
vkQueueSubmit(..., fences[cur]);
vkQueuePresentKHR(...);
```

Abridged from src/render/vulkan/SDL_render_vulkan.c (lines 392, 3785–3805, 5144–5147), program test/testsprite.

2 The recorded run as one happens-before order: CPU and GPU queue



3 Diagnostic, verbatim checker output (first of 166 in 20 frames)

```
[timing] timing.cross-queue-race: 'buffer#1' is written on 'host' and read on 'graphics0' with
nothing ordering the two.
    at vkCmdDraw @seq 1139 (command_buffer#1, submitted @seq 1227 on graphics0): vertex buffer 0
    note: other access: host write to buffer#1 (mapped memory#1 +0x0), before call seq 1442
    note: add a semaphore (GPU queues) or wait on the frame's fence (host) between them
```

Synchronization validation on the same program and frames: no message. It does not track host writes to mapped memory.

4 The fix, re-checked

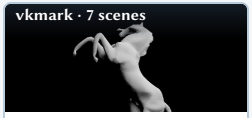
```
// one set of vertex buffers per frame in flight
SDL_memcpy(vertexBuffers[cur][0].mapped,
           vertices, size);
checker: all checks passed
```

An accepted trace is race-free under **every** device schedule consistent with its recorded synchronization, for the modelled accesses on the recorded host path, host writes included.

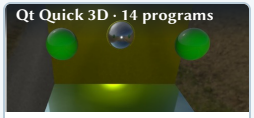
Programs



this race · **patched: 166 → 0**



5 causes · **patched: 1,124 → 0**



study 1 · **3 true causes**



study 2 · **3 true causes**



study 2 · **1 true cause**